

Target return else -1

```
#include <stdio.h>
```

```
int main()
{
    int arr[20], n, target;
    int low, high, mid, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements in ascending order:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Enter target: ");
    scanf("%d", &target);

    low = 0;
    high = n - 1;

    while(low <= high)
    {
        mid = (low + high) / 2;
        if(arr[mid] == target)
        {
            printf("taget found at Index = %d\n", mid);
            return 0;
        }
        else if(arr[mid] < target)
        {
            low = mid + 1;
        }
        else
        {
            high = mid - 1;
        }
    }

    printf("Index = -1\n");

    return 0;
}
```

Zero index

```
#include <stdio.h>
int main()
{
    int nums[20], n, target, i, j;

    printf("Enter size of array: ");
    scanf("%d", &n);

    printf("Enter %d elements: ", n);

    for (i = 0; i < n; i++)
        scanf("%d", &nums[i]);

    printf("Enter target: ");
    scanf("%d", &target);

    // Sort the array
    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (nums[j] > nums[j + 1])
            {
                int temp = nums[j];
                nums[j] = nums[j + 1];
                nums[j + 1] = temp;
            }
        }
    }

    // Print sorted array
    printf("Sorted array: ");

    for (i = 0; i < n; i++){
        printf("%d ", nums[i]);
    }
    // Find target indices
    int found = 0;

    for (i = 0; i < n; i++)
    {
        if (nums[i] == target)
        {

            printf("\nTarget Indices found at %d", i);
            found = 1;
        }
    }
    if (found == 0)
        printf("target not found in the array\n");

    return 0;
}
```

Tower

```
#include <stdio.h>

void towerOfHanoi(int n, char source, char auxiliary, char destination){
    if (n == 1){
        printf("Move disk 1 from %c to %c\n", source, destination);
        return;
    }
    towerOfHanoi(n - 1, source, destination, auxiliary);
    printf("Move disk %d from %c to %c\n", n, source, destination);
    towerOfHanoi(n - 1, auxiliary, source, destination);
}

int main(){
    int n;
    printf("Enter number of disks: ");
    scanf("%d", &n);
    printf("\nSteps to solve Tower of Hanoi:\n");
    towerOfHanoi(n, 'A', 'B', 'C');
    return 0;
}
```

Growth

```
#include <stdio.h>
#include <math.h>

int main()
{
    int n;

    printf("Enter value of n: ");
    scanf("%d", &n);

    printf("\nGrowth of Functions for n = %d\n", n);

    printf("O(1)    = 1\n");
    printf("O(log n)  = %.2f\n", log2(n));
    printf("O(n)     = %d\n", n);
    printf("O(n log n) = %.2f\n", n * log2(n));
    printf("O(n^2)    = %d\n", n * n);
    printf("O(2^n)    = %.0f\n", pow(2, n));

    return 0;
}
```

Quick

```
#include <stdio.h>

int partition(int arr[], int low, int high){
    int pivot = arr[high];
    int i = low - 1;
    int j, temp;

    for (j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;

            temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int p = partition(arr, low, high);

        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}

int main()
{
    int arr[100], n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    quickSort(arr, 0, n - 1);

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

    return 0;
}
```

Min max

```
#include <stdio.h>
```

```
#define SIZE 20
```

```
int main()
```

```
{  
    int a[SIZE];  
    int min, max;  
    int i, j, count;
```

```
    printf("Enter 20 elements:\n");
```

```
    for (i = 0; i < SIZE; i++)  
        scanf("%d", &a[i]);
```

```
    // Find minimum and maximum
```

```
    if (a[0] < a[1])
```

```
    {  
        min = a[0];  
        max = a[1];
```

```
    }
```

```
    else
```

```
    {  
        min = a[1];  
        max = a[0];
```

```
    }
```

```
    i = 2;
```

```
    while (i < SIZE - 1)
```

```
    {  
        int localMin, localMax;
```

```
        if (a[i] < a[i + 1])
```

```
        {  
            localMin = a[i];  
            localMax = a[i + 1];
```

```
        }
```

```
        else
```

```
        {  
            localMin = a[i + 1];  
            localMax = a[i];
```

```
        }
```

```
        if (localMin < min)
```

```
            min = localMin;
```

```
        if (localMax > max)
```

```
            max = localMax;
```

```
        i += 2;
```

```
    }
```

```
    printf("\nMinimum element = %d", min);
```

```
printf("\nMaximum element = %d\n", max);
```

```
// Find frequency
```

```
for (i = 0; i < SIZE; i++)
```

```
{  
    count = 1;
```

```
    // Check if element was already counted
```

```
    for (j = 0; j < i; j++)
```

```
    {  
        if (a[i] == a[j])  
        {  
            count = 0;  
            break;  
        }  
    }
```

```
    if (count != 0)
```

```
    {  
        count = 1;
```

```
        for (j = i + 1; j < SIZE; j++)
```

```
        {  
            if (a[i] == a[j])  
                count++;  
        }
```

```
        printf("%d \nFrequency of each element-> %d times\n", a[i], count);
```

```
    }  
}
```

```
return 0;
```

```
}
```